

BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: C Batch: 2 Practical No: 5

Roll no: 62 PRN No: 12411517 Name of Student: Akshada Lohakare

Subject Name : Deep Learning

Problem Statement : . Implement and compare RNN, LSTM, and GRU models for sequence classification, and analyze their performance using appropriate evaluation metrics.

Algorithm :

1. Load the Dataset
2. Load the DAIR.AI Emotion dataset containing text samples and six emotion classes.
3. Prepare the Data
4. Separate the dataset into training, validation, and testing sets.
5. Preprocess the Text
6. Tokenize the text and convert words into numerical sequences. Pad the sequences to a fixed length.
7. Build the RNN Model
8. Create an Embedding layer followed by a Simple RNN layer and a Softmax output layer.
9. Build the LSTM Model
10. Create an Embedding layer followed by an LSTM layer and a Softmax output layer.
11. Build the GRU Model
12. Create an Embedding layer followed by a GRU layer and a Softmax output layer.
13. Train the Models
14. Train all three models using the training data and use the validation data to monitor their performance.

15. Test the Models
16. Use the unseen test dataset to generate predictions from RNN, LSTM, and GRU.
17. Evaluate Performance
18. Calculate Accuracy, Precision, Recall, and F1-score for each model.
19. Compare the Models
20. Compare the evaluation metrics and analyze the performance of RNN, LSTM, and GRU using a results table and confusion matrix.

Code :

```
# 1. import libraries

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from datasets import load_dataset

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, SimpleRNN, LSTM, GRU,
Dense, Dropout
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences

from sklearn.metrics import accuracy_score, precision_score,
recall_score
from sklearn.metrics import f1_score, confusion_matrix,
classification_report

# 2. load dataset

# Load the Emotion dataset directly from Hugging Face.
# It contains predefined train, validation and test sets.

dataset = load_dataset("dair-ai/emotion")

train_df = pd.DataFrame(dataset["train"])
val_df = pd.DataFrame(dataset["validation"])
```

```
test_df = pd.DataFrame(dataset["test"])

print("Train:", train_df.shape)
print("Validation:", val_df.shape)
print("Test:", test_df.shape)

# 3. PREPARE DATA

# Emotion labels:
# 0 - sadness
# 1 - joy
# 2 - love
# 3 - anger
# 4 - fear
# 5 - surprise

label_names = [
    "sadness",
    "joy",
    "love",
    "anger",
    "fear",
    "surprise"
]

X_train_text = train_df["text"].values
y_train = train_df["label"].values

X_val_text = val_df["text"].values
y_val = val_df["label"].values

X_test_text = test_df["text"].values
y_test = test_df["label"].values

# 4. text tokenization and padding

# Convert text into numerical sequences.
# The tokenizer vocabulary is created using only training data.

max_words = 10000
max_length = 100

tokenizer = Tokenizer(
```

```

        num_words=max_words,
        oov_token="<OOV>"
    )

tokenizer.fit_on_texts(X_train_text)

X_train = tokenizer.texts_to_sequences(X_train_text)
X_val = tokenizer.texts_to_sequences(X_val_text)
X_test = tokenizer.texts_to_sequences(X_test_text)

# Make all sequences the same length.

X_train = pad_sequences(
    X_train,
    maxlen=max_length,
    padding="post",
    truncating="post"
)

X_val = pad_sequences(
    X_val,
    maxlen=max_length,
    padding="post",
    truncating="post"
)

X_test = pad_sequences(
    X_test,
    maxlen=max_length,
    padding="post",
    truncating="post"
)

print("Processed training shape:", X_train.shape)
print("Processed validation shape:", X_val.shape)
print("Processed testing shape:", X_test.shape)

# 5. function to build and evaluate models

def evaluate_model(model, name):

    model.compile(
        optimizer="adam",
        loss="sparse_categorical_crossentropy",

```

```
        metrics=["accuracy"]
    )

    print("\nTraining", name, "...")

    history = model.fit(
        X_train,
        y_train,
        validation_data=(X_val, y_val),
        epochs=10,
        batch_size=64,
        verbose=1
    )

    # Evaluate on test data
    loss, accuracy = model.evaluate(
        X_test,
        y_test,
        verbose=0
    )

    # Generate predictions
    predictions = model.predict(
        X_test,
        verbose=0
    )

    predicted_classes = np.argmax(
        predictions,
        axis=1
    )

    # Calculate evaluation metrics
    precision = precision_score(
        y_test,
        predicted_classes,
        average="weighted"
    )

    recall = recall_score(
        y_test,
        predicted_classes,
        average="weighted"
    )
```

```

        f1 = f1_score(
            y_test,
            predicted_classes,
            average="weighted"
        )

    print("\n", name, "Results")
    print("-----")
    print("Accuracy :", round(accuracy, 4))
    print("Precision:", round(precision, 4))
    print("Recall   :", round(recall, 4))
    print("F1 Score :", round(f1, 4))

    return history, predicted_classes, accuracy, precision, recall,
f1

# 6. simple rnn model

rnn_model = Sequential([
    Embedding(max_words, 64),
    SimpleRNN(64),
    Dropout(0.3),
    Dense(6, activation="softmax")
])

rnn_history, rnn_pred, rnn_acc, rnn_precision, rnn_recall, rnn_f1 = \
    evaluate_model(rnn_model, "RNN")

# 7. lstm model

lstm_model = Sequential([
    Embedding(max_words, 64),
    LSTM(64),
    Dropout(0.3),
    Dense(6, activation="softmax")
])

lstm_history, lstm_pred, lstm_acc, lstm_precision, lstm_recall,
lstm_f1 = \
    evaluate_model(lstm_model, "LSTM")

```

```

# 8. GRU MODEL

gru_model = Sequential([
    Embedding(max_words, 64),
    GRU(64),
    Dropout(0.3),
    Dense(6, activation="softmax")
])

gru_history, gru_pred, gru_acc, gru_precision, gru_recall, gru_f1 = \
    evaluate_model(gru_model, "GRU")

# 9. COMPARE ALL THREE MODELS

results = pd.DataFrame({
    "Model": ["RNN", "LSTM", "GRU"],
    "Accuracy": [rnn_acc, lstm_acc, gru_acc],
    "Precision": [rnn_precision, lstm_precision, gru_precision],
    "Recall": [rnn_recall, lstm_recall, gru_recall],
    "F1 Score": [rnn_f1, lstm_f1, gru_f1]
})

print("\nMODEL COMPARISON")
print("=====")
print(results.to_string(index=False))

# 10. ACCURACY COMPARISON GRAPH

# Compare validation accuracy of RNN, LSTM and GRU.

plt.figure(figsize=(9, 5))

plt.plot(
    rnn_history.history["val_accuracy"],
    label="RNN"
)

plt.plot(
    lstm_history.history["val_accuracy"],
    label="LSTM"
)

```

```

plt.plot(
    gru_history.history["val_accuracy"],
    label="GRU"
)

plt.xlabel("Epoch")
plt.ylabel("Validation Accuracy")
plt.title("RNN vs LSTM vs GRU - Validation Accuracy")

plt.legend()
plt.show()

# 11. confusion matrix

# Display the confusion matrix for the best-performing model.
# Here, the model with the highest F1-score is selected.

best_index = results["F1 Score"].idxmax()
best_model_name = results.loc[best_index, "Model"]

if best_model_name == "RNN":
    best_predictions = rnn_pred

elif best_model_name == "LSTM":
    best_predictions = lstm_pred

else:
    best_predictions = gru_pred

cm = confusion_matrix(
    y_test,
    best_predictions
)

plt.figure(figsize=(8, 6))

sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    xticklabels=label_names,
    yticklabels=label_names
)

```

```
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title(best_model_name + " Confusion Matrix")

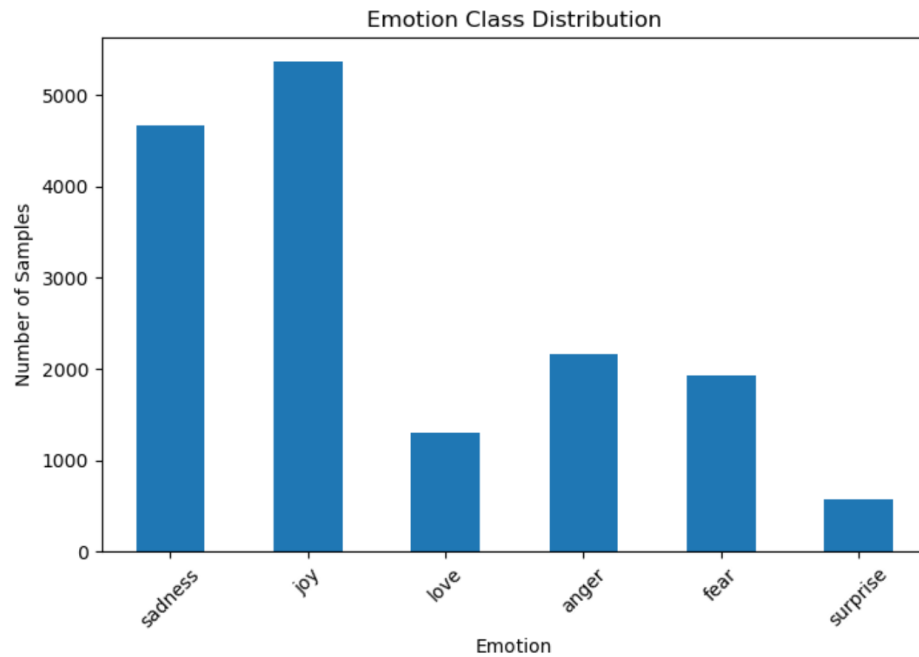
plt.show()

# 12. classification report

print("\nClassification Report -", best_model_name)
print(
    classification_report(
        y_test,
        best_predictions,
        target_names=label_names
    )
)
```

Output :

- **Emotion Class distribution**



- **Model layers**

Model: "sequential"

Layer (type)	Output Shape	Param #
embedding (Embedding)	?	0 (unbuilt)
simple_rnn (SimpleRNN)	?	0 (unbuilt)
dropout (Dropout)	?	0
dense (Dense)	?	0 (unbuilt)

Total params: 0 (0.00 B)

Trainable params: 0 (0.00 B)

Non-trainable params: 0 (0.00 B)

- Model layers:

Model: "sequential_2"

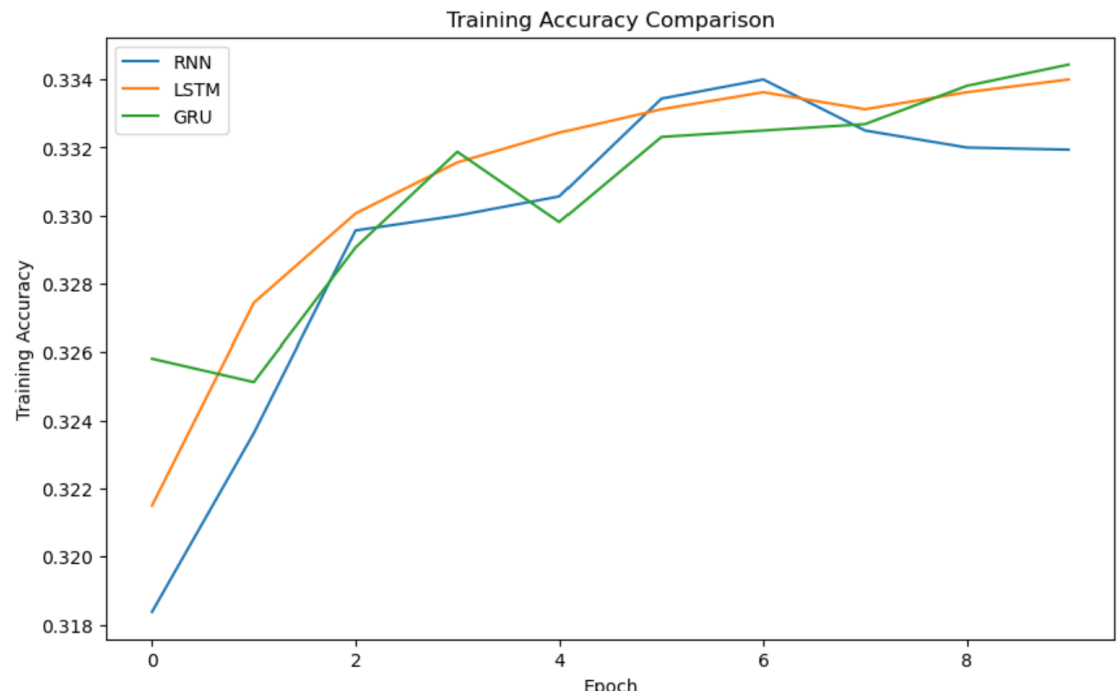
Layer (type)	Output Shape	Param #
embedding_2 (Embedding)	?	0 (unbuilt)
lstm_1 (LSTM)	?	0 (unbuilt)
dropout_2 (Dropout)	?	0
dense_2 (Dense)	?	0 (unbuilt)

Total params: 0 (0.00 B)

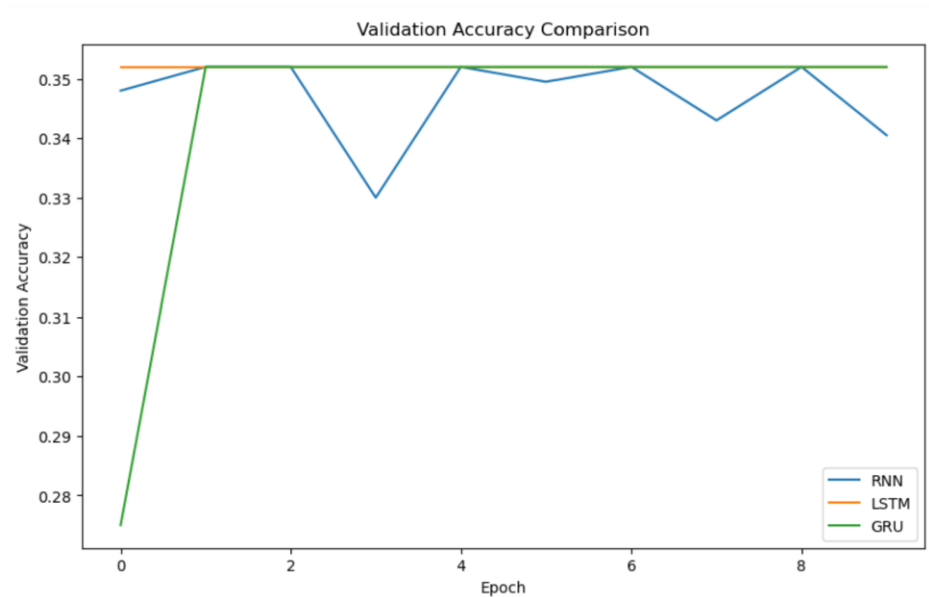
Trainable params: 0 (0.00 B)

Non-trainable params: 0 (0.00 B)

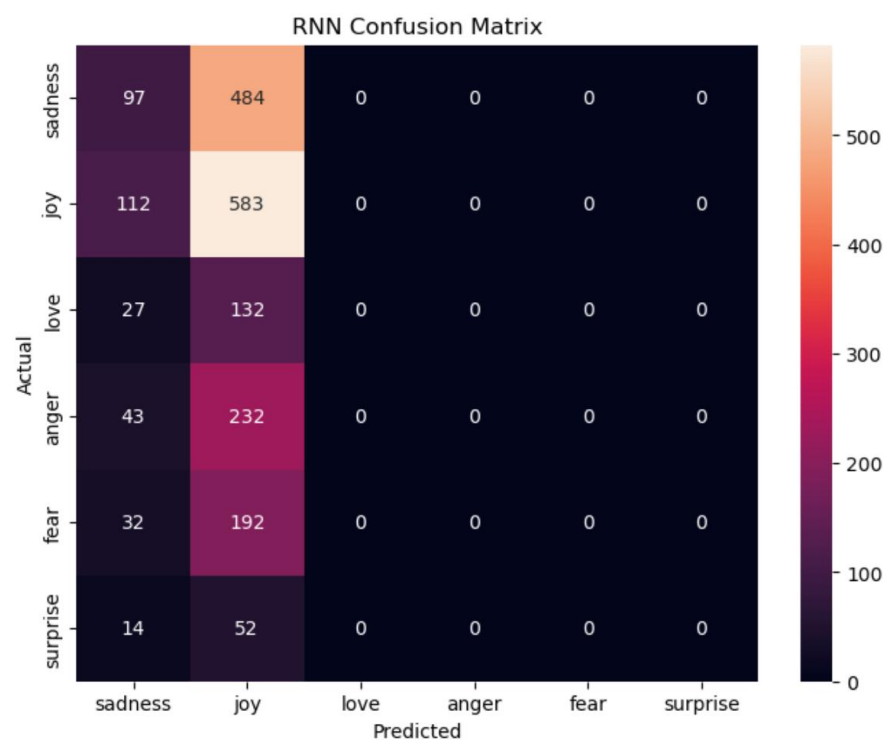
- Training Accuracy Comparison Visualization



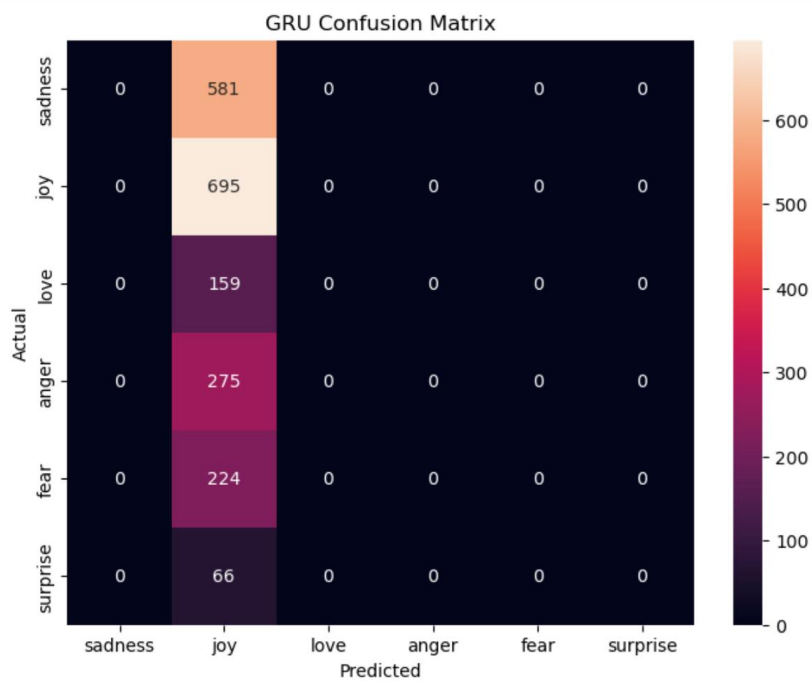
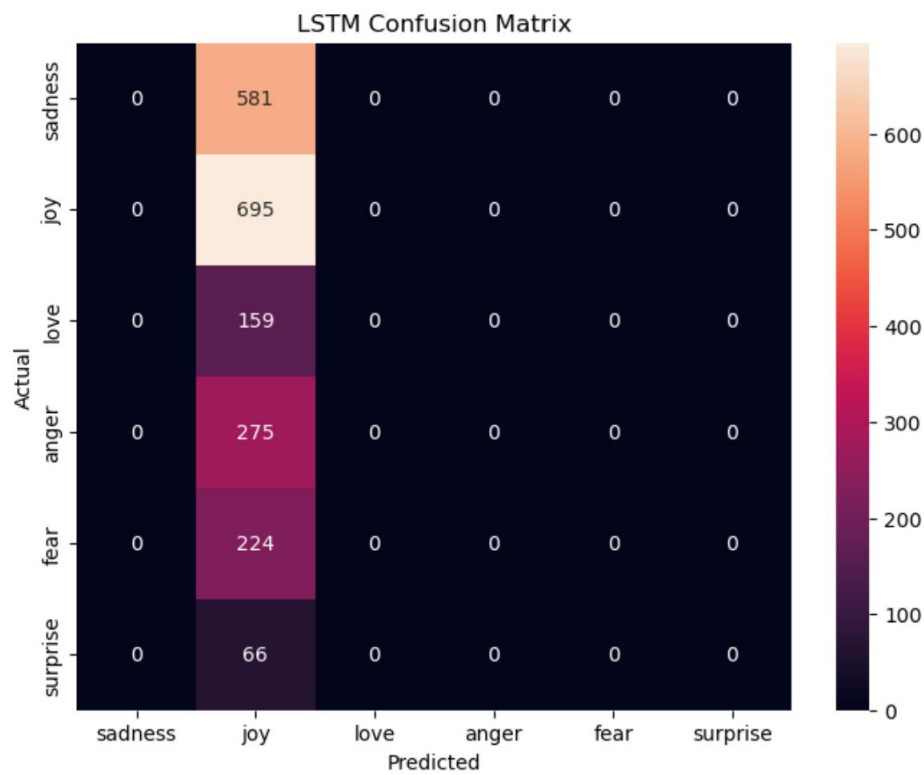
- Validation Accuracy Comparison Visualization



- RNN Confusion matrix



- LSTM Confusion Matrix



RNN Classification Report

	precision	recall	f1-score	support
sadness	0.30	0.17	0.21	581
joy	0.35	0.84	0.49	695
love	0.00	0.00	0.00	159
anger	0.00	0.00	0.00	275
fear	0.00	0.00	0.00	224
surprise	0.00	0.00	0.00	66
accuracy			0.34	2000
macro avg	0.11	0.17	0.12	2000
weighted avg	0.21	0.34	0.23	2000

LSTM Classification Report

	precision	recall	f1-score	support
sadness	0.00	0.00	0.00	581
joy	0.35	1.00	0.52	695
love	0.00	0.00	0.00	159
anger	0.00	0.00	0.00	275
fear	0.00	0.00	0.00	224
surprise	0.00	0.00	0.00	66
accuracy			0.35	2000
macro avg	0.06	0.17	0.09	2000
weighted avg	0.12	0.35	0.18	2000

GRU Classification Report

	precision	recall	f1-score	support
sadness	0.00	0.00	0.00	581
joy	0.35	1.00	0.52	695
love	0.00	0.00	0.00	159
anger	0.00	0.00	0.00	275
fear	0.00	0.00	0.00	224
surprise	0.00	0.00	0.00	66
accuracy			0.35	2000
macro avg	0.06	0.17	0.09	2000
weighted avg	0.12	0.35	0.18	2000
